

Procesamiento de Datos en Series Temporales Multimodales

Reference Guide Técnica — FatigueSet Biometría

Análisis de Windowing, Data Leakage, Sincronización y Normalización

2026

Aviso

Documento técnico que sintetiza el análisis práctico de los notebooks:

- `windowing_data_leakage.ipynb`
- `sincronizacion_normalizacion.ipynb`

Incluye espacios designados [IMG] para inserción de figuras principales.

Índice

1. Introducción	4
2. Conceptos Fundamentales	4
2.1. Series Temporales en Biometría	4
2.2. Frecuencia de Muestreo y Aliasing	4
3. Windowing y Extracción de Features	5
3.1. Definición Formal	5
3.2. Sliding sin Solapamiento vs Con Solapamiento	5
3.2.1. Sin solapamiento	5
3.2.2. Con solapamiento	5
3.3. Pseudocódigo: Crear Ventanas	6
4. Data Leakage: Cuatro Fuentes de Contaminación	7
4.1. Leakage 1: Split Temporal Incorrecto	7
4.1.1. El Problema	7
4.1.2. La Solución	7
4.1.3. Resultados Cuantitativos	7
4.2. Leakage 2: Normalización Global	7
4.2.1. El Problema	7
4.2.2. La Solución	7
4.2.3. Impacto Cuantitativo	8
4.3. Leakage 3: Overlapping + Split Aleatorio	8
4.3.1. El Problema	8
4.3.2. Similitud Coseno	8
4.3.3. Resultados	8
4.4. Leakage 4: Entre Sujetos (LOSO)	8
4.4.1. El Problema	8
4.4.2. La Solución: Leave-One-Subject-Out	9
4.4.3. Correctitud en LOSO	9
4.5. Checklist Anti-Leakage	9
5. Sincronización Multimodal	10
5.1. El Problema de los Relojes Distintos	10
5.2. Pipeline de Sincronización	10
5.3. Resampling	10
5.3.1. Downsampling	10
5.3.2. Upsampling	10
5.4. Interpolación	11
5.4.1. A) Interpolación Lineal	11
5.4.2. B) Interpolación Spline Cúbica	11
5.4.3. C) Forward Fill (Zero-Order Hold)	11
5.4.4. Comparación Empírica	11
5.5. Pipeline Completo	12
5.5.1. Resultado del Tensor	12
6. Normalización de Datos	13
6.1. Normalización Global	13
6.1.1. Definición	13
6.1.2. Interpretación	13

6.1.3.	Ventajas y Riesgos	13
6.1.4.	Resultados en FatigueSet	13
6.2.	Normalización por Sujeto	14
6.2.1.	Definición	14
6.2.2.	Interpretación	14
6.2.3.	Ventajas y Riesgos	14
6.2.4.	Medias por Sujeto (Original)	14
6.3.	LOSO sin Data Leakage	14
6.3.1.	Procedimiento	14
6.3.2.	Verificación (FatigueSet)	15
7.	Figuras Referenciadas (del Notebook)	16
7.1.	Figura 1: Desalineación Temporal	16
7.2.	Figura 2: Downsampling EDA	16
7.3.	Figura 3: Métodos de Interpolación	17
7.4.	Figura 4: Tensor Sincronizado	18
7.5.	Figura 5: Normalización Global	18
7.6.	Figura 6: Split Temporal	19
8.	Conclusiones y Buenas Prácticas	20
8.1.	Resumen de Operaciones	20
8.2.	Verificación Pre-Entrenamiento	20
8.3.	Recomendaciones para TFG	20
A.	Funciones Auxiliares	21
A.1.	Similitud Coseno	21
A.2.	Features Básicos	21
A.3.	Validación Split Temporal	21
B.	Referencias a Notebooks	22

1. Introducción

El procesamiento de datos de series temporales multimodales en biometría es una tarea crítica para:

1. Evitar artefactos y ruido estructural
2. Prevenir **data leakage**: contaminación información train $\leftrightarrow$ test
3. Sincronizar sensores que operan a distintas frecuencias
4. Normalizar correctamente sin sesgos

Este documento proporciona:

- **Teoría**: Ecuaciones formales y conceptos fundamentales
- **Práctica**: Código Python y pseudocódigo ejecutable
- **Datos reales**: Valores del dataset FatigueSet (12 sujetos, 3 sesiones)

Nota: Los ejemplos numéricos provienen directamente de ejecuciones en los notebooks referenciados.

2. Conceptos Fundamentales

2.1. Series Temporales en Biometría

Una serie temporal es una secuencia de observaciones ordenadas en el tiempo:

$$\mathbf{x} = [x_1, x_2, \dots, x_n] \quad \text{donde} \quad x_t \in \mathbb{R}, \quad t \in \{1, \dots, n\} \quad (1)$$

En biometría multimodal, se adquieren **múltiples señales simultáneamente**:

Cuadro 1: Sensores en FatigueSet (P01/S01 — 26.3 minutos)

Sensor	Filas	Hz estimado	Duración (min)
Chest HR (Zephyr)	1578	1.00	26.3
EDA (Empatica E4)	6313	4.00	26.3
EEG Alpha (Muse)	14400	9.13	26.3

Problema fundamental: Aunque todas duran 26.3 minutos, cada sensor opera con su propio reloj interno. El índice de fila i en cada señal *no corresponde al mismo instante temporal*. Esto requiere sincronización explícita.

2.2. Frecuencia de Muestreo y Aliasing

Piensa en muestrear una señal como hacer fotos a algo que se mueva. Si

La frecuencia de muestreo es el número de muestras por segundo:

$$f_s = \frac{\text{número de muestras}}{\text{duración en segundos}} \quad (2)$$

Según el teorema de Nyquist, la frecuencia máxima que puede capturarse sin aliasing es $f_s/2$.

Implicación: Si EEG se muestrea a ≈ 256 Hz, pero downsampling a 1 Hz descartaría frecuencias $> 0,5$ Hz. Para señales fisiológicas (variabilidad cardiaca, ritmos cerebrales) esto puede ser problemático.

3. Windowing y Extracción de Features

3.1. Definición Formal

Windowing divide una serie temporal de longitud n en m segmentos de tamaño k :

$$W_i = \{x_j : \text{inicio}_i \leq j < \text{inicio}_i + k\}, \quad i = 1, \dots, m \quad (3)$$

donde el inicio de cada ventana es:

$$\text{inicio}_i = i \cdot \text{step_size} \quad (4)$$

El número de ventanas resultantes:

$$m = \left\lfloor \frac{n - k}{\text{step_size}} \right\rfloor + 1 \quad (5)$$

Cada ventana se transforma en **features escalares** (estadísticos):

$$\mu_i = \frac{1}{k} \sum_{j \in W_i} x_j \quad (\text{media}) \quad (6)$$

$$\sigma_i = \sqrt{\frac{1}{k} \sum_{j \in W_i} (x_j - \mu_i)^2} \quad (\text{desviación}) \quad (7)$$

$$\min_i = \min(W_i), \quad \max_i = \max(W_i) \quad (\text{extremos}) \quad (8)$$

3.2. Sliding sin Solapamiento vs Con Solapamiento

3.2.1. Sin solapamiento

$$\text{step_size} = k \implies m = \lfloor n/k \rfloor \quad (9)$$

Las ventanas son disjuntas. Con $n = 1578$ muestras y $k = 100$: $m = 15$ ventanas.

3.2.2. Con solapamiento

$$\text{step_size} < k \implies m = \left\lfloor \frac{n - k}{\text{step_size}} \right\rfloor + 1 \quad (10)$$

Con $k = 100$ y $\text{step_size} = 50$ (50 % overlap): $m = 30$ ventanas ($2 \times$ más).

Trade-off:

- Sin overlap: Menos muestras, pero sin correlación espuria
- Con overlap: Más muestras, pero alta correlación temporal entre ventanas consecutivas

El solapamiento introduce un riesgo crítico si el split train/test es **aleatorio**: los datos correlacionados pueden filtrarse mutuamente. Esto es **data leakage**.

3.3. Pseudocódigo: Crear Ventanas

Listing 1: Algoritmo de windowing

```
1 def crear_ventanas(serie, window_size, step_size):
2     """Segmenta serie temporal en ventanas."""
3     ventanas = []
4
5     for inicio in range(0, len(serie) - window_size + 1, step_size):
6         fin = inicio + window_size
7         ventana = {
8             'indices': (inicio, fin),
9             'timestamp_inicio': timestamps[inicio],
10            'timestamp_fin': timestamps[fin - 1],
11            'valores': serie[inicio:fin]
12        }
13        ventanas.append(ventana)
14
15    return ventanas
16
17 # Ejemplo
18 n_ventanas = crear_ventanas(hr_data, window_size=100, step_size=50)
19 print(f"Total_ventanas: {len(n_ventanas)}")
```

4. Data Leakage: Cuatro Fuentes de Contaminación

Data leakage ocurre cuando **información del conjunto de test filtra al modelo durante entrenamiento**, invalidando validación experimental.

4.1. Leakage 1: Split Temporal Incorrecto

4.1.1. El Problema

En series temporales, un split **aleatorio** mezcla pasado y futuro:

$$\text{Aleatorio (MAL)} : \quad \text{train} = \{W_1, W_3, W_4, \dots\}, \quad \text{test} = \{W_2, W_5, \dots\} \quad (11)$$

Si train contiene indicador posterior a test, el modelo conoce el futuro”.

4.1.2. La Solución

Split **temporal** con límite claro:

$$\text{Split en } t_c : \quad \text{train} = \{(t_i, x_i) : t_i < t_c\}, \quad \text{test} = \{(t_i, x_i) : t_i \geq t_c\} \quad (12)$$

Además, hacer windowing **después del split**, no antes.

4.1.3. Resultados Cuantitativos

En P01/S01 con 30 ventanas (overlap 50%) y split aleatorio:

- **95.8 %** de ventanas train inician después de la primera ventana test
- Contaminación severa: información temporal entrelazada

Split temporal correcto (en índice 1262 de 1578):

- Train: 24 ventanas, Test: 5 ventanas
- **0 %** de solapamiento de índices entre folds

4.2. Leakage 2: Normalización Global

4.2.1. El Problema

Normalización Z-score típica calcula media y desviación con **todos los datos**:

$$\mu_{\text{ALL}} = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{ALL}}} x_i \quad (13)$$

Esto incluye test. Resultado: información de test contamina el normalizador.

4.2.2. La Solución

Calcular estadísticas **solo con datos de entrenamiento**:

$$\mu_{\text{TRAIN}} = \frac{1}{n_{\text{train}}} \sum_{i \in \text{TRAIN}} x_i, \quad \sigma_{\text{TRAIN}} = \sqrt{\frac{1}{n_{\text{train}}} \sum_{i \in \text{TRAIN}} (x_i - \mu_{\text{TRAIN}})^2} \quad (14)$$

Luego normalizar ambos conjuntos:

$$z_i = \frac{x_i - \mu_{\text{TRAIN}}}{\sigma_{\text{TRAIN}}} \quad \forall i \in \text{TRAIN} \cup \text{TEST} \quad (15)$$

4.2.3. Impacto Cuantitativo

Comparando normalización global vs train-only en FatigueSet:

Cuadro 2: Diferencia en valores test normalizados: global vs train-only

Feature	$ \Delta $ promedio
max	0.0387
std	0.0360
mean	0.0348
range	0.0332
min	0.0125

Aunque pequeño individualmente, acumula error en 100+ muestras.

4.3. Leakage 3: Overlapping + Split Aleatorio

4.3.1. El Problema

Con solapamiento alto ($\text{step} \ll k$), ventanas consecutivas comparten $> 50\%$ índices:

$$W_i = [x_1, \dots, x_{100}], \quad W_{i+1} = [x_{51}, \dots, x_{150}] \quad (16)$$

Si split es aleatorio, train y test comparten esos 50 índices comunes.

4.3.2. Similitud Coseno

Imagina dos vectores como flechas en el espacio. La similitud coseno no se fija en lo largas que son, sino en hacia

La similitud coseno entre dos vectores mide su alineación:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}| \cdot |\mathbf{v}|} \quad (17)$$

Valores: $[-1, 1]$ (típicamente $[0, 1]$ para datos no-negativos).

4.3.3. Resultados

En P01/S01:

- Split aleatorio: similitud máxima $\approx 0,9979$, solapamiento $\approx 50\%$
- Split temporal post-windowing: similitud $\approx 0,9983$, solapamiento $\approx 50\%$ (frontera contaminada)
- Split temporal correcto: similitud $\approx 0,9979$, solapamiento $= 0\%$ (sin contaminación)

Conclusión: El solapamiento de índices es más crítico que la similitud coseno.

Los tres casos parecen similares (sim ≈ 0.998)
Pero solo uno es válido
La diferencia real no está en la similitud... sino en si comparten

4.4. Leakage 4: Entre Sujetos (LOSO)

4.4.1. El Problema

En biometría, si el mismo sujeto aparece en train y test, el modelo aprende **identidad** (quién es) en lugar de **estado** (cómo está: fatiga, estrés).

En cada iteración, dejas un sujeto completo fuera para test y entrenas con el resto.

4.4.2. La Solución: Leave-One-Subject-Out

Para cada sujeto p :

$$\text{FOLD}_p : \quad \text{TRAIN}_p = \{x_i : s_i \neq p\}, \quad \text{TEST}_p = \{x_i : s_i = p\} \quad (18)$$

Esto valida que el modelo generaliza a **nuevos sujetos** sin datos previos.

4.4.3. Correctitud en LOSO

Si usamos normalización por-sujeto en LOSO:

$$\mu_p^{\text{TRAIN}} = \frac{1}{n_p^{\text{TRAIN}}} \sum_{i \in \text{TRAIN}, s_i = p} x_i \quad (19)$$

La media μ_p se calcula **solo con datos del fold de entrenamiento**, nunca test.

Véase Tabla 3: simulación LOSO en FatigueSet (108 filas = 12 sujetos $\times$ 9 muestras/sujeto).

Cuadro 3: Distribución LOSO: puntos por fold

Sujeto Test	n_train	n_test
Cualquiera	99	9

4.5. Checklist Anti-Leakage

Antes de entrenar, verificar:

- ✓ **Split antes de windowing:** Dividir datos brutos en train/test por tiempo/sujeto primero
- ✓ **Normalización train-only:** μ, σ calculadas **solamente** del conjunto entrenamiento
- ✓ **Evitar overlap train-test:** Verificar que no hay solapamiento de índices entre folds
- ✓ **Features sin visión al futuro:** Ningún feature depende de valores posteriores a su ventana
- ✓ **LOSO para biometría:** Validar con leave-one-subject-out, no random cross-validation
- ✓ **Recalcular normalizadores por fold:** Cada fold en LOSO tiene sus propios μ, σ de TRAIN

5. Sincronización Multimodal

5.1. El Problema de los Relojes Distintos

Cada sensor registra timestamps independientes:

```

1 # Chest HR - 1 muestra/segundo
2 t_chest = [10:53:43, 10:53:44, 10:53:45, ...]
3
4 # EDA - 4 muestras/segundo
5 t_eda = [10:53:43.00, 10:53:43.25, 10:53:43.50, ...]
6
7 # EEG - ~256 Hz
8 t_eeg = [10:53:43.0039, 10:53:43.0078, 10:53:43.0117, ...]
```

Los **índices de fila no son equivalentes** entre sensores. Índice 100 en HR $\neq$ índice 100 en EDA $\neq$ índice 100 en EEG.

5.2. Pipeline de Sincronización

La sincronización requiere 5 pasos:

1. **Indexar por datetime:** Usar timestamp como índice, no número de fila
2. **Elegir frecuencia objetivo:** Típicamente, la del sensor más lento
3. **Resampling:** Cambiar la frecuencia de cada señal
4. **Interpolación:** Rellenar valores faltantes
5. **Concatenación:** Fusionar en tensor 2D sincronizado

Frecuencia objetivo en FatigueSet: 1 Hz (dictada por HR del chest, el más lento).

5.3. Resampling

5.3.1. Downsampling

Reducir Hz mediante agregación (típicamente media):

$$y[t] = \text{MEAN}(\{x[t + k \cdot \Delta t] : k = 0, 1, \dots, m\}) \quad (20)$$

Ejemplo: EDA de 4 Hz a 1 Hz

Cuadro 4: Downsampling EDA: 4 Hz $\rightarrow$ 1 Hz

Original	6313 muestras
Downsampled (1 Hz)	1578 muestras
Ratio reducción	$\approx 4\times$

5.3.2. Upsampling

Aumentar Hz creando posiciones temporales vacías (NaN):

```

1 # Pseudocódigo
2 hr_original = [70, 72, 65, 68, ...] # 1 muestra/s
3 hr_250ms = [70, NaN, NaN, NaN, 72, NaN, NaN, NaN, 65, ...]
4 # 4 muestras/s: 3 NaN nuevos por cada valor original
```

En FatigueSet, upsampling HR de 1 Hz a 250 ms (4 Hz):

Cuadro 5: Upsampling HR: 1 Hz $\rightarrow$ 250 ms (4 Hz)

Original	1578 muestras
Upsampled (250ms)	6309 posiciones
Valores conocidos	1578
NaN generados	4731 (75.0 %)

5.4. Interpolación

Las posiciones vacías (NaN) deben rellenarse. Métodos comunes:

5.4.1. A) Interpolación Lineal

Conecta puntos conocidos con recta:

$$y_{\text{interp}}[t] = y[t_i] + \frac{t - t_i}{t_{i+1} - t_i} (y[t_{i+1}] - y[t_i]) \quad (21)$$

Donde $t_i \leq t < t_{i+1}$ son dos timestamps adyacentes con valores.

5.4.2. B) Interpolación Spline Cúbica

Usa polinomios cúbicos por tramos, garantizando continuidad C^2 :

$$y_{\text{spline}}(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3 \quad (22)$$

para $t \in [t_i, t_{i+1}]$.

5.4.3. C) Forward Fill (Zero-Order Hold)

Mantiene el último valor:

$$y_{\text{fill}}[t] = y[t_i] \quad \text{para todo } t \in [t_i, t_{i+1}) \quad (23)$$

Útil para eventos discretos; introduce escaleras en señales continuas.

5.4.4. Comparación Empírica

En HR upsampling (5 minutos):

Cuadro 6: Métodos interpolación: características

Método	Propiedad
Lineal	Cambios de pendiente abruptos
Spline	Curva suave, natural
Forward fill	Escaleras; para eventos discretos

Recomendación: Spline cúbica para señales fisiológicas continuas.

5.5. Pipeline Completo

Listing 2: Pipeline sincronización multimodal

```

1  FREQ_OBJETIVO = '1s' # 1 Hz
2
3  def sincronizar_senal(dataframe, columna, freq=FREQ_OBJETIVO):
4      # 1. Indexar por datetime
5      ts = dataframe.set_index('datetime')[columna]\
6          .sort_index().dropna()
7
8      # 2. Resampling a frecuencia objetivo
9      resampled = ts.resample(freq).mean()
10
11     # 3. Interpolacion + ffill para rellenos finales
12     interpolated = resampled.interpolate('linear')\
13         .ffill().bfill()
14
15     return interpolated
16
17 # Aplicar a cada sensor
18 hr_sync = sincronizar_senal(df_chest, 'hr')
19 eda_sync = sincronizar_senal(df_wrist, 'eda')
20 eeg_mean = df_eeg[CANALES_EEG].mean(axis=1)
21 eeg_sync = sincronizar_senal(df_eeg_con_media, 'eeg_mean')
22
23 # 4. Concatenacion en tensor sincronizado
24 tensor = pd.concat([hr_sync, eda_sync, eeg_sync],
25                    axis=1).dropna()
26
27 # Resultado: [T x 3] donde T = 1578 instantes
28 print(f"Tensor: {tensor.shape}") # (1578, 3)

```

5.5.1. Resultado del Tensor

Cuadro 7: Ejemplo tensor sincronizado (10 filas)

Timestamp	HR (bpm)	EDA (μ S)	EEG alpha (dB)
2021-08-19 10:53:43	64.0000	0.0705	0.4308
2021-08-19 10:53:44	62.0000	0.0695	0.4114
2021-08-19 10:53:45	59.0000	0.0705	0.2709
2021-08-19 10:53:46	59.0000	0.0698	0.2819
⋮	⋮	⋮	⋮

Ahora cada fila corresponde al mismo instante temporal para las tres modalidades.

6. Normalización de Datos

6.1. Normalización Global

6.1.1. Definición

Z-score calculado sobre **todo el dataset**:

$$\mu_{\text{global}} = \frac{1}{N} \sum_{i=1}^N x_i \quad (24)$$

$$\sigma_{\text{global}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu_{\text{global}})^2} \quad (25)$$

$$z_i^{\text{global}} = \frac{x_i - \mu_{\text{global}}}{\sigma_{\text{global}}} \quad (26)$$

6.1.2. Interpretación

Mantiene **diferencias basales entre individuos**. La información de nivel fisiológico permanece codificada.

Ejemplo: HR en reposo (Persona A: 55 bpm vs Persona B: 85 bpm).

Con $\mu_{\text{global}} = 87,88$ bpm, $\sigma_{\text{global}} = 20,16$:

$$z_A = \frac{55 - 87,88}{20,16} = -1,63 \quad (27)$$

$$z_B = \frac{85 - 87,88}{20,16} = -0,14 \quad (28)$$

La diferencia relativa permanece: $\Delta z \approx 1,5$.

6.1.3. Ventajas y Riesgos

Cuadro 8: Normalización global: trade-offs

Ventajas	Riesgos
Mantiene diferencias inter-sujeto	Modelo aprende identidad
Generaliza a nuevos sujetos	Sensible a outliers
Escala única	Confunde "quién" con "cómo"

6.1.4. Resultados en FatigueSet

Parámetros globales (108 filas $\times$ 3 features):

Cuadro 9: Estadísticas normalización global

Feature	μ_{global}	σ_{global}
HR (bpm)	87.8750	20.1570
EDA (μS)	1.2730	1.7890
HRV (ms)	69.4640	31.1550

Post-normalización: media ≈ 0 , std ≈ 1 en todas las features.

6.2. Normalización por Sujeto

6.2.1. Definición

Z-score calculado **dentro de cada sujeto**:

$$\mu_p = \frac{1}{n_p} \sum_{i \in \text{sujeto } p} x_i \quad (29)$$

$$z_i^{(p)} = \frac{x_i - \mu_p}{\sigma_p} \quad \text{para todo } i \in \text{sujeto } p \quad (30)$$

6.2.2. Interpretación

Elimina diferencias basales. Cada sujeto queda centrado en el mismo espacio estadístico, focalizándose en **cambios relativos**.

Ejemplo:

- Persona A: HR reposo 55 → esfuerzo 75 (cambio +20)
- Persona B: HR reposo 85 → esfuerzo 105 (cambio +20)

Ambos reflejan el mismo cambio relativo, eliminando identidad.

6.2.3. Ventajas y Riesgos

Cuadro 10: Normalización por-sujeto: trade-offs

Ventajas	Riesgos
Detecta cambios relativos	No generaliza a nuevos sujetos
Reduce sesgo por identidad	Oculto diferencias poblacionales
Mejora modelos LOSO	Requiere datos suficientes

6.2.4. Medias por Sujeto (Original)

HR antes de normalizar - diferencias basales:

Cuadro 11: Media HR por participante (antes normalizar)

Sujeto	HR (bpm)	Sujeto	HR (bpm)
1	72.63	7	100.49
2	89.18	8	92.20
3	100.45	9	107.54
4	78.93	10	91.99
5	81.05	11	75.83
6	81.64	12	82.58

Rango: 72.63 a 107.54 bpm (35 bpm de diferencia basal).

Post-normalización por-sujeto: todas las medias ≈ 0 , todas las desv. est. ≈ 1 .

6.3. LOSO sin Data Leakage

6.3.1. Procedimiento

Para m sujetos en validación leave-one-subject-out:

Listing 3: LOSO correcto sin leakage

```

1 def LOSO_validacion(dataset):
2     """Validacion LOSO con normalizacion train-only."""
3
4     resultados = []
5
6     for sujeto_test in dataset.sujeto.unique():
7         # 1. Split por sujeto
8         train = dataset[dataset.sujeto != sujeto_test]
9         test = dataset[dataset.sujeto == sujeto_test]
10
11         # 2. Calcular normalizador SOLO con TRAIN
12         mu_train = train.mean()
13         sigma_train = train.std()
14
15         # 3. Aplicar a ambos con mismos params
16         train_norm = (train - mu_train) / sigma_train
17         test_norm = (test - mu_train) / sigma_train
18
19         # 4. Entrenar con train, evaluar en test
20         modelo.fit(train_norm.features, train_norm.target)
21         metrica = modelo.evaluate(test_norm.features,
22                                   test_norm.target)
23
24         resultados.append({
25             'sujeto': sujeto_test,
26             'metrica': metrica,
27             'mu_train': mu_train
28         })
29
30     return resultados

```

Punto crítico: μ y σ se recalculan en **cada fold**, solo con TRAIN.

6.3.2. Verificación (FatigueSet)

Simulación LOSO con normalización global:

Cuadro 12: LOSO: parámetros independientes por fold

Sujeto test	n_train	n_test	μ_{train} HR	σ_{train} HR
1	99	9	89.2610	20.1566
2	99	9	87.7560	20.1565
3	99	9	86.7320	20.1560
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
12	99	9	88.3570	20.1550

Nótese: μ_{train} varía ligeramente (89.26 vs 87.76) entre folds, reflejando que cada fold tiene composición distinta. **Nunca** usamos test para calcular estos parámetros.

7. Figuras Referenciadas (del Notebook)

7.1. Figura 1: Desalineación Temporal



Figura 1: Desalineación temporal en P01/S01: cada sensor (HR, EDA, EEG) opera con su propia frecuencia y los índices de fila no representan el mismo instante.

7.2. Figura 2: Downsampling EDA

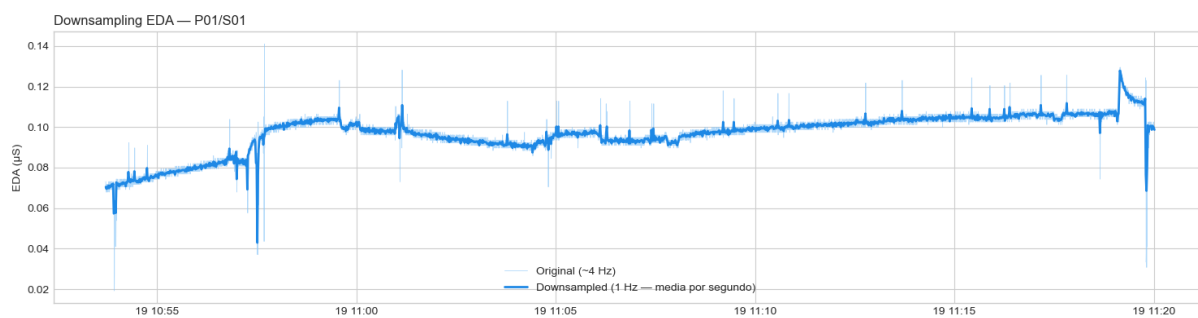


Figura 2: Downsampling de EDA en P01/S01: comparación entre señal original (~ 4 Hz) y señal agregada a 1 Hz mediante media por segundo.

7.3. Figura 3: Métodos de Interpolación

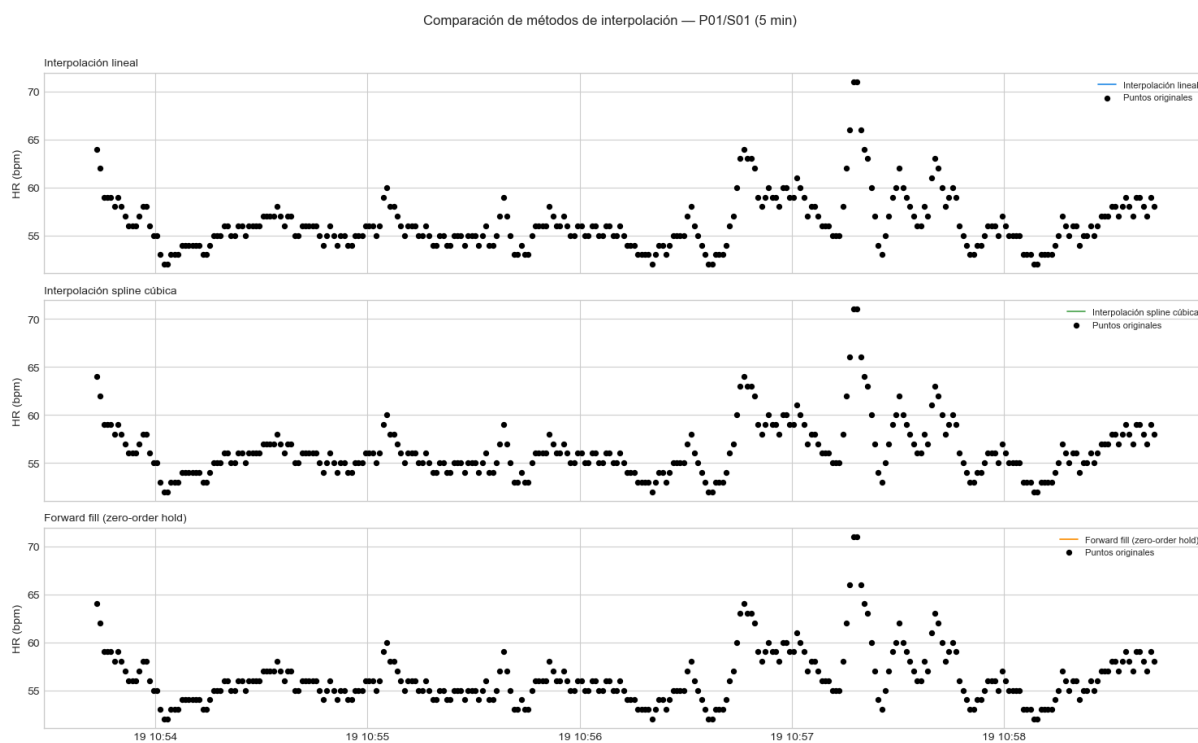


Figura 3: Comparación de interpolación en HR upsampled (5 minutos): lineal, spline cúbica y forward fill, junto con los puntos originales.

7.4. Figura 4: Tensor Sincronizado



Figura 4: Tensor multimodal sincronizado a 1 Hz para P01/S01: HR, EDA y EEG Alpha alineados sobre un eje temporal común.

7.5. Figura 5: Normalización Global

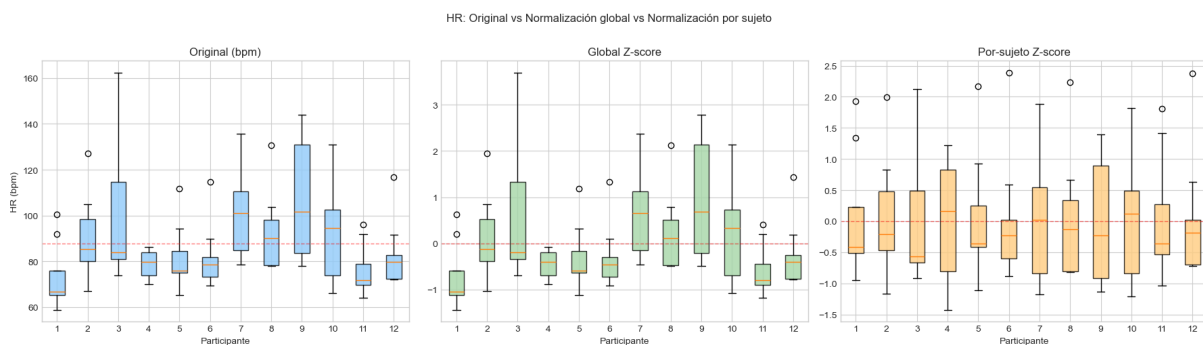


Figura 5: Comparación de HR original, normalización global y normalización por sujeto; se aprecia el efecto sobre la distribución por participante.

7.6. Figura 6: Split Temporal

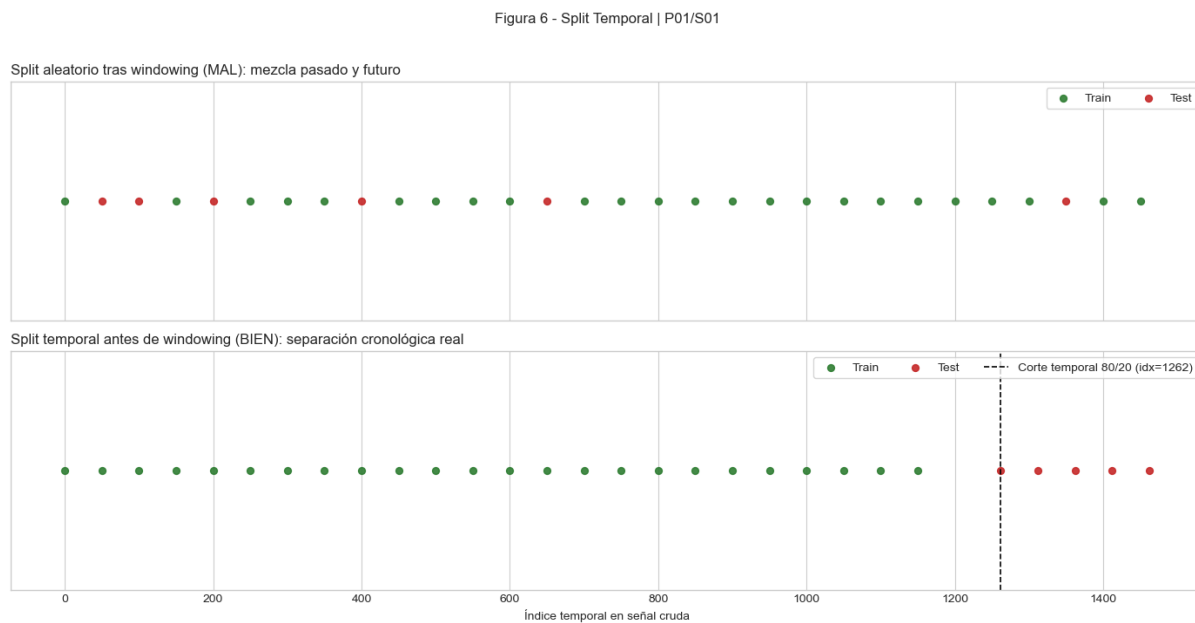


Figura 6: Comparación de esquemas de partición en P01/S01: split aleatorio tras windowing (incorrecto) frente a split temporal antes de windowing (correcto).

8. Conclusiones y Buenas Prácticas

8.1. Resumen de Operaciones

Cuadro 13: Referencia rápida: técnicas y herramientas

Técnica	Qué hace	Pandas/Scipy
Windowing	Segmenta serie en bloques	loop + slicing
Downsampling	Reduce Hz por agregación	<code>.resample().mean()</code>
Upsampling	Aumenta Hz (crea NaN)	<code>.resample().asfreq()</code>
Interp. lineal	Rellena NaN lineal	<code>.interpolate('linear')</code>
Interp. spline	Rellena NaN curva suave	<code>.interpolate('cubic')</code>
Norm. global	Z-score todo dataset	<code>(x - x.mean())/x.std()</code>
Norm. sujeto	Z-score por persona	<code>.groupby().transform()</code>
LOSO	Validación sin leakage	Loop + recalcular μ/σ

8.2. Verificación Pre-Entrenamiento

Checklist final:

1. Forma tensor: (T, M) donde T = instantes, M = modalidades
2. Alineación temporal: Índices t corresponden al mismo instante real
3. Sin NaN: Matriz completa después interpolación
4. Estadísticas post-norm: $\mu \approx 0$, $\sigma \approx 1$
5. Sin leakage temporal: Split train/test tiene frontera clara
6. Sin leakage normalización: $\mu_{\text{norm}}, \sigma_{\text{norm}}$ solo de TRAIN
7. LOSO para biometría: Ningún sujeto en $\text{train} \cap \text{test}$

8.3. Recomendaciones para TFG

- **Priorizar LOSO:** Validación leave-one-subject-out es obligatoria en biometría
- **Documentar split:** Describir explícitamente temporal vs aleatorio
- **Reportar diagnostics:** Incluir tablas de leakage checks
- **Normalización conservadora:** Si dudas, usar por-sujeto para LOSO
- **Reproducibilidad:** Fijar random seeds, versionar scripts

Nota importante: Si las métricas caen después de corregir leakage, *no es fracaso*. Es validez experimental recuperada.

A. Funciones Auxiliares

A.1. Similitud Coseno

Listing 4: Similitud coseno

```
1 import numpy as np
2
3 def cosine_similarity(u, v):
4     """Similitud coseno: [-1, 1]."""
5     norm_u = np.linalg.norm(u)
6     norm_v = np.linalg.norm(v)
7
8     if norm_u == 0 or norm_v == 0:
9         return 0.0
10
11     return float(np.dot(u, v) / (norm_u * norm_v))
12
13 # Ejemplo
14 w1 = np.array([1.2, 3.4, 5.6])
15 w2 = np.array([1.3, 3.3, 5.7])
16 sim = cosine_similarity(w1, w2)
17 print(f"Similitud: {sim:.4f}") # ~0.9999
```

A.2. Features Básicos

Listing 5: Extracción de features estadísticos

```
1 def calcular_features(ventana):
2     """Features estadísticos de una ventana."""
3     return {
4         'media': float(np.mean(ventana)),
5         'std': float(np.std(ventana)),
6         'min': float(np.min(ventana)),
7         'max': float(np.max(ventana)),
8         'rango': float(np.max(ventana) - np.min(ventana)),
9     }
10
11 # Ejemplo
12 ventana = np.array([60, 62, 61, 63, 65, 64])
13 feats = calcular_features(ventana)
14 print(feats)
```

A.3. Validación Split Temporal

Listing 6: Validar split temporal sin leakage

```
1 def validar_split_temporal(df_train, df_test,
2                             col_tiempo='datetime'):
3     """Verifica ausencia de solapamiento temporal."""
4     t_train_max = df_train[col_tiempo].max()
5     t_test_min = df_test[col_tiempo].min()
6
7     if t_test_min <= t_train_max:
8         print("ERROR: Solapamiento temporal!")
9         print(f"TRAIN max: {t_train_max}")
10        print(f"TEST min: {t_test_min}")
```

```
11         return False
12     else:
13         print("OK: Split temporal valido")
14         return True
```

B. Referencias a Notebooks

■ windowing_data_leakage.ipynb:

- Análisis de leakage tipos 1-3
- Similitud coseno entre ventanas
- Comparación split aleatorio vs temporal

■ sincronizacion_normalizacion.ipynb:

- Carga multimodal
- Resampling e interpolación
- Normalizacion global vs por-sujeto
- LOSO sin data leakage